Lab program-32:

32. Construct a C program to simulate the Least Recently Used paging technique of memory management.

Code:

```
#include <stdio.h>

#define MAX_FRAMES 3

void printFrames(int frames[], int n)
{
    for (int i = 0; i < n; i++)
    {
        if (frames[i] == -1)
            printf(" - ");
        else
            printf(" %d ", frames[i]);
    }

    printf("\n");
}

int main()
{
    int frames[MAX_FRAMES];
    int usageHistory[MAX_FRAMES];
    int pageFaults = 0;

    int referenceString[] =
        {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};

    int n = sizeof(referenceString) / sizeof(referenceString[0]);

    // Initialize frames and usage history
    for (int i = 0; i < MAX_FRAMES; i++)
    {
        frames[i] = -1;
        usageHistory[i] = 0;
    }

    printf("Reference String: ");

    for (int i = 0; i < n; i++)
        printf("%d ", referenceString[i]);

    printf("\n\nPage Replacement Order:\n");
```

```c
for (int i = 0; i < n; i++)
{
    int page = referenceString[i];
    int pageFound = 0;

    // Check if page is already in memory
    for (int j = 0; j < MAX_FRAMES; j++)
    {
        if (frames[j] == page)
        {
            pageFound = 1;

            // Update usage history
            for (int k = 0; k < MAX_FRAMES; k++)
            {
                if (k != j && frames[k] != -1)
                    usageHistory[k]++;
            }

            usageHistory[j] = 0;
            break;
        }
    }

    // Page fault
    if (!pageFound)
    {
        printf("Page %d -> ", page);

        // Find least recently used page
        int lruPage = 0;

        for (int j = 1; j < MAX_FRAMES; j++)
        {
            if (usageHistory[j] > usageHistory[lruPage])
                lruPage = j;
        }

        int replacedPage = frames[lruPage];

        frames[lruPage] = page;
        usageHistory[lruPage] = 0;

        if (replacedPage != -1)
            printf("Replace %d with %d: ",
                   replacedPage, page);
        else
```

```c
                printf("Load into an empty frame: ");

            printFrames(frames, MAX_FRAMES);

            pageFaults++;

            // Increment usage of other pages
            for (int j = 0; j < MAX_FRAMES; j++)
            {
                if (j != lruPage && frames[j] != -1)
                    usageHistory[j]++;
            }
        }
    }

    printf("\nTotal Page Faults: %d\n", pageFaults);

    return 0;
}
```

OUTPUT:

```
PS C:\Users\SMILE\Documents\OS\Lab programs> ./lab_32.exe
Reference String: 7 0 1 2 0 3 0 4 2 3 0 3 2

Page Replacement Order:
Page 7 -> Load into an empty frame:  7  -  -
Page 0 -> Replace 7 with 0:  0  -  -
Page 1 -> Replace 0 with 1:  1  -  -
Page 2 -> Replace 1 with 2:  2  -  -
Page 0 -> Replace 2 with 0:  0  -  -
Page 3 -> Replace 0 with 3:  3  -  -
Page 0 -> Replace 3 with 0:  0  -  -
Page 4 -> Replace 0 with 4:  4  -  -
Page 2 -> Replace 4 with 2:  2  -  -
Page 3 -> Replace 2 with 3:  3  -  -
Page 0 -> Replace 3 with 0:  0  -  -
Page 3 -> Replace 0 with 3:  3  -  -
Page 2 -> Replace 3 with 2:  2  -  -

Total Page Faults: 13
```